

Computer Vision Lab 02 - Code Cheat Sheet

How to use this sheet in the coding exam

For every code block, first identify:

- INPUT: what image/video/data the code receives
- OPERATION: what transformation/function is doing the CV work
- OUTPUT: image, histogram, mask, fused image, or enhanced frame
- PARAMETERS: what you would change if the question changes

Lab 2 syllabus priority

Core photometric / point-transformation material

1. Logarithmic transformation
2. Gamma / Power-law transformation
3. Piecewise transformation
4. Histogram
5. Histogram Equalization

Task-specific OpenCV material

6. Image loading and display
7. Thresholding
8. False-color mapping (`COLORMAP_JET`)
9. Color-balance channel adjustment
10. Weighted image fusion (`cv2.addWeighted`)
11. Video capture and frame-by-frame processing
12. Side-by-side display with `np.hstack`

Source note: The Lab 2 task sheet specifies the three tasks and the operations to perform. It does not give fixed numerical values for the color-balance coefficients, threshold, or gamma in the task statements, so those values are marked as parameters/examples here rather than being presented as instructor-specified constants.

1. Basic image input and display

Purpose: Read an image, optionally as grayscale, check that it loaded, display it, and close the window.

```
import cv2

# Read the image from the given path.
# cv2.IMREAD_GRAYSCALE makes the image single-channel (0-255 intensity values).
image = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Always check whether OpenCV successfully loaded the image.
if image is None:
    print("Error: Image could not be loaded.")
    exit()

# Display the image in an OpenCV window.
cv2.imshow("Image", image)

# Wait until any key is pressed before continuing.
cv2.waitKey(0)

# Close all OpenCV windows.
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.imread(filename, flags)`
 - `filename` = image file path
 - `flags` = how to read image
 - common values:
 - `cv2.IMREAD_GRAYSCALE` = grayscale / 1 channel
 - `cv2.IMREAD_COLOR` = color image / 3 channels (BGR)
- `cv2.imshow(window_name, image)`
 - `window_name` = title of display window
 - `image` = image/array to show
- `cv2.waitKey(delay)`
 - `delay` = milliseconds to wait
 - `0` = wait indefinitely
- `cv2.destroyAllWindows()`
 - closes all OpenCV display windows

Exam modifications

- Change path: `"data/xray2.png"`
- Read color: `cv2.imread("data/image.png", cv2.IMREAD_COLOR)`
- Use `cv2.waitKey(1)` inside video loops rather than `0`

2. Histogram - calculate and display

Purpose: A histogram counts how many pixels occur at each intensity level. For an 8-bit grayscale image there are 256 intensity levels (0-255).

```
import cv2
import matplotlib.pyplot as plt

# Read the image as grayscale because we want a grayscale intensity histogram.
gray = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Calculate a histogram with 256 bins for intensities 0-255.
# [gray] = list containing the input image
# [0] = channel 0 (the only channel in grayscale)
# None = no mask; use every pixel
# [256] = number of histogram bins
# [0, 256] = intensity range
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

# Plot the histogram.
plt.plot(hist)
plt.title("Grayscale Histogram")
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.xlim([0, 256])
plt.show()
```

Function / argument dictionary

- `cv2.calcHist(images, channels, mask, histSize, ranges)`
 - `images` = list of input images, e.g. `[gray]`
 - `channels` = channel indices, e.g. `[0]`
 - `mask` = mask selecting pixels; `None` means all pixels
 - `histSize` = number of bins, e.g. `[256]`
 - `ranges` = intensity range, e.g. `[0, 256]`
- `plt.plot(x_or_y)` = plot histogram values as a line
- `plt.title(text)` = graph title
- `plt.xlabel(text)` = x-axis label
- `plt.ylabel(text)` = y-axis label
- `plt.xlim(left, right)` = x-axis limits
- `plt.show()` = display graph

Exam recognition

Histogram -> tells how pixel intensities are distributed.
It does NOT tell you where those pixels are located spatially.

Exam modifications

- 256 grayscale levels: `histSize=[256]`, `ranges=[0,256]`
- For a color histogram, calculate channels separately or convert to another color space.

3. Histogram Equalization

Purpose: Improve contrast by redistributing intensity values based on the image histogram.

```
import cv2

# Read a single-channel grayscale image.
gray = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Redistribute grayscale intensities to improve contrast.
equalized = cv2.equalizeHist(gray)

# Show original and equalized images.
cv2.imshow("Original", gray)
cv2.imshow("Histogram Equalized", equalized)

# Wait for a key and close windows.
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.equalizeHist(src)`
 - `src` = input **single-channel 8-bit** grayscale image
 - returns = histogram-equalized image

Exam recognition

Low contrast / poor intensity spread
-> `equalizeHist()`
-> better use of available intensity range

Important

`cv2.equalizeHist()` is normally used on a grayscale single-channel 8-bit image in this workflow. Do not blindly pass a 3-channel BGR image.

Exam modifications

- Change image variable: `equalized = cv2.equalizeHist(xray)`
- Equalize CT and MRI independently:
 - `ct_eq = cv2.equalizeHist(ct)`
 - `mri_eq = cv2.equalizeHist(mri)`

4. Logarithmic Transformation

Purpose: Expand dark intensity values and compress bright intensity values. Useful when faint/dark details need to become more visible.

Formula:

$$s = c * \log(1 + r)$$

```
import cv2
import numpy as np

# Read grayscale image.
image = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Find the largest input intensity so we can scale the output into 0-255.
max_value = np.max(image)

# Protect against a completely black image, where log(1 + 0) = 0.
if max_value > 0:
    c = 255 / np.log(1 + max_value)
else:
    c = 1

# Apply logarithmic transformation to every pixel.
# +1 prevents taking log(0).
log_image = c * np.log(1 + image)

# Convert the floating-point result back to 8-bit image format.
log_image = np.uint8(log_image)

# Display the result.
```

```
cv2.imshow("Log Transformation", log_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `np.max(array)`
 - returns the maximum value in the array
- `np.log(array)`
 - element-wise natural logarithm
- `np.uint8(array)`
 - converts values to unsigned 8-bit integer representation
- `1 + image`
 - avoids the undefined `log(0)` case

Exam recognition

Log -> expands dark values -> compresses bright values

Exam modifications

- Change input variable: `image`, `xray`, `fused`, etc.
- Keep the same transformation formula unless the question gives a different mapping.

5. Gamma / Power-Law Transformation

Purpose: Control brightness/tonal response with a power curve.

Formula:

$$s = c * r^{\text{gamma}}$$

In the common normalized 0-1 implementation, use:

```
gamma < 1 -> brighten
gamma = 1 -> approximately unchanged
gamma > 1 -> darken / suppress bright values
```

```
import cv2
import numpy as np
```

```

# Read grayscale image.
image = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Set gamma according to the question.
# Example: gamma < 1 brightens darker/mid-tone regions.
gamma = 0.5

# Normalize pixel values from 0-255 to 0-1.
normalized = image / 255.0

# Apply the power-law transformation.
gamma_image = np.power(normalized, gamma)

# Convert the normalized result back to 0-255 and store as uint8.
gamma_image = np.uint8(gamma_image * 255)

# Display the result.
cv2.imshow("Gamma Transformation", gamma_image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Function / argument dictionary

- `np.power(base, exponent)`
 - `base` = normalized pixel values
 - `exponent` = `gamma`
- `image / 255.0`
 - converts 8-bit intensity range to approximately 0-1
- `* 255`
 - converts normalized result back to display range
- `np.uint8(...)`
 - converts to 8-bit image format

Exam modifications

- `gamma = 0.5` or another value `< 1` -> brighter
- `gamma = 1.0` -> unchanged
- `gamma = 2.0` or another value `> 1` -> darker

6. Piecewise Transformation - Contrast Stretching

Purpose: Apply different rules to different ranges of input intensities. A common example is contrast stretching: map a low input range to a wider output range.

```
import cv2
import numpy as np

# Read the grayscale image.
image = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Input breakpoints: intensity values r1 and r2.
r1 = 50
r2 = 150

# Desired output breakpoints: s1 and s2.
s1 = 0
s2 = 255

# Start with an output array having the same size as the input.
piecewise_image = np.zeros_like(image, dtype=np.uint8)

# Region 1: intensities below r1 are mapped to s1.
piecewise_image[image < r1] = s1

# Region 2: linearly stretch intensities between r1 and r2.
mask = (image >= r1) & (image <= r2)
piecewise_image[mask] = np.uint8(
    ((s2 - s1) / (r2 - r1)) * (image[mask] - r1) + s1
)

# Region 3: intensities above r2 are mapped to 255.
piecewise_image[image > r2] = 255

# Display result.
cv2.imshow("Piecewise / Contrast Stretching", piecewise_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `np.zeros_like(image, dtype=...)`
 - creates an output array with the same shape as `image`
 - `dtype=np.uint8` makes it an 8-bit image
- `mask = condition`
 - creates a Boolean selection of pixels meeting a condition
- `image[mask]`
 - selects only the pixels where `mask` is `True`

Exam recognition

Piecewise -> different formula/rule for different intensity ranges.

Simpler piecewise threshold form

```
# Keep only intensities >= T.  
_, binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
```

Thresholding is itself a piecewise intensity rule.

7. Thresholding

Purpose: Separate pixels according to a threshold. For binary thresholding, values below the threshold become 0 and values at/above it become the maximum value.

```
import cv2  
  
# Read grayscale image.  
gray = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)  
  
# Set the threshold according to the question.  
threshold = 180  
  
# Apply binary thresholding.  
# gray          = input image  
# threshold     = cutoff value  
# 255           = output value for pixels that pass the threshold  
# THRESH_BINARY = binary threshold rule  
_, binary = cv2.threshold(  
    gray,  
    threshold,  
    255,  
    cv2.THRESH_BINARY  
)  
  
# Display the binary mask.  
cv2.imshow("Thresholded", binary)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.threshold(src, thresh, maxval, type)`
 - `src` = input image
 - `thresh` = threshold value
 - `maxval` = value assigned to pixels that pass the rule
 - `type` = thresholding method, e.g. `cv2.THRESH_BINARY`
 - `returns` = (`threshold_used`, `output_image`)

Exam recognition

```
pixel < T    -> 0
pixel >= T   -> 255
```

Exam modifications

- Change `threshold` value if Sir gives a new cutoff.
- Another common form is inverse binary: `cv2.THRESH_BINARY_INV`

8. False-Color Mapping - COLORMAP_JET

Purpose: Convert a grayscale intensity image into a false-color heatmap so intensity differences can be visually easier to distinguish.

```
import cv2

# Read and/or obtain a grayscale image first.
gray = cv2.imread("data/image.png", cv2.IMREAD_GRAYSCALE)

# Convert grayscale intensities into a JET false-color heatmap.
heatmap = cv2.applyColorMap(
    gray,
    cv2.COLORMAP_JET
)

# Display the false-color image.
cv2.imshow("JET Heatmap", heatmap)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.applyColorMap(src, colormap)`

- `src` = 8-bit grayscale image in this workflow
- `colormap` = color map, e.g. `cv2.COLORMAP_JET`
- `returns` = 3-channel color image

Exam recognition

Grayscale intensity -> colormap -> false-color visualization

Exam modifications

- Use the exact map named in the question.
- Task 1 and Task 3 explicitly call for `COLORMAP_JET`.

9. Color Balance by Channel Scaling

Purpose: Apply a mathematical color-balance adjustment by changing the strength of B, G, and R channels.

The Lab 2 task sheet asks for a mathematical color-balance shift but does not specify exact channel coefficients. Therefore the coefficients below are placeholders/examples that should be changed to match the question.

```
import cv2
import numpy as np

# `heatmap` should be a 3-channel BGR image.
heatmap = cv2.applyColorMap(
    gray,
    cv2.COLORMAP_JET
)

# OpenCV stores color images as Blue, Green, Red (BGR).
b, g, r = cv2.split(heatmap)

# Channel scaling factors - CHANGE THESE if the question gives values.
b_scale = 0.90
g_scale = 1.00
r_scale = 0.90

# Multiply each channel and keep all values within 0-255.
b = np.clip(b * b_scale, 0, 255).astype(np.uint8)
g = np.clip(g * g_scale, 0, 255).astype(np.uint8)
r = np.clip(r * r_scale, 0, 255).astype(np.uint8)
```

```
# Merge the corrected channels back into a BGR image.  
balanced = cv2.merge([b, g, r])
```

Corrected code indentation:

```
b_scale = 0.90  
g_scale = 1.00  
r_scale = 0.90
```

Function / argument dictionary

- `cv2.split(image)`
 - `image` = multi-channel image
 - `returns` = one array per channel
- `np.clip(array, min_value, max_value)`
 - keeps values inside the specified range
- `.astype(np.uint8)`
 - converts array to 8-bit integer format
- `cv2.merge([b, g, r])`
 - combines channels in the given order
 - for OpenCV color images, `[b, g, r]` is BGR order

Exam modifications

- Change `b_scale`, `g_scale`, `r_scale`.
- If Sir gives an additive shift instead of multiplication, use channel addition and clip to 0-255.

10. Weighted Image Fusion - `cv2.addWeighted`

Purpose: Combine two aligned images using weights.

Formula:

```
dst = alpha * src1 + beta * src2 + gamma
```

```
import cv2
```

```
# `image1` and `image2` must have compatible dimensions/types for direct fusion.
```

```
# Weight assigned to the first image.
alpha = 0.70

# Weight assigned to the second image.
beta = 0.30

# Scalar brightness offset added to the result.
gamma_offset = 0

# Blend the two images.
fused = cv2.addWeighted(
    image1,
    alpha,
    image2,
    beta,
    gamma_offset
)
```

Function / argument dictionary

- `cv2.addWeighted(src1, alpha, src2, beta, gamma)`
 - `src1` = first input image
 - `alpha` = weight for first image
 - `src2` = second input image
 - `beta` = weight for second image
 - `gamma` = scalar added after weighted sum
 - returns = fused image

Exam recognition

```
src1 * alpha + src2 * beta + gamma
```

Exam modifications

- CT heavier, MRI lighter:
 - e.g. `alpha=0.70`, `beta=0.30`
- Use whatever weights the question gives.

11. Side-by-Side Image Comparison with `np.hstack`

Purpose: Put images next to each other for visual comparison.

```
import numpy as np

# All images should have compatible heights/channels for horizontal stacking.
comparison = np.hstack((image1, image2, image3))
```

Function / argument dictionary

- `np.hstack(tup)`
 - `tup` = tuple/list of arrays with matching dimensions except horizontally combined axis
 - `returns` = horizontally concatenated array

Exam recognition

```
(image1, image2, image3) -> np.hstack -> one wide comparison image
```

Exam modification

- Two images: `np.hstack((raw, enhanced))`
- Three images: `np.hstack((ct, mri, fused))`

12. Video Capture Setup

Purpose: Open a video file and read it frame by frame.

```
import cv2

# Open the provided video file.
# This is a video file, not a webcam.
cap = cv2.VideoCapture("data/echo.mp4")

# Check whether OpenCV successfully opened the video.
if not cap.isOpened():
    print("Error: Could not open video.")
    exit()
```

Function / argument dictionary

- `cv2.VideoCapture(source)`
 - `source` = video file path or camera index
 - file example: `"data/echo.mp4"`
 - webcam example: `0`
- `cap.isOpened()`
 - returns `True` if the capture source opened successfully

Exam recognition

```
Video file -> cv2.VideoCapture("path.mp4")
Webcam -> cv2.VideoCapture(0)
```

13. Frame-by-Frame Video Loop

Purpose: Continuously read and process each video frame.

```
import cv2

cap = cv2.VideoCapture("data/echo.mp4")

while True:
    # Read the next video frame.
    ret, frame = cap.read()

    # Stop if no frame was returned (usually end of video).
    if not ret:
        break

    # Process `frame` here.

    # Press q to quit before the video naturally ends.
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Release the video source.
cap.release()

# Close all OpenCV windows.
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cap.read()`

- `returns (ret, frame)`
- `ret = success flag`
- `frame = current frame`
- `cap.release()`
 - releases the video capture resource
- `ord('q')`
 - converts character `q` into its integer key code
- `cv2.waitKey(1)`
 - waits briefly so video can continue playing and keyboard input can be detected

Critical exam pattern

```
ret, frame = cap.read()
if not ret:
    break
```

14. Convert a Color Video Frame to Grayscale

Purpose: Convert a normal BGR frame into a single-channel grayscale intensity image before histogram equalization.

```
# Convert BGR frame to grayscale.
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Function / argument dictionary

- `cv2.cvtColor(src, code)`
 - `src` = source image
 - `code` = color-space conversion code
 - here: `cv2.COLOR_BGR2GRAY`

Exam recognition

```
BGR frame -> cvtColor(..., COLOR_BGR2GRAY) -> grayscale frame
```

15. MASTER TASK 1 - Diagnostic Enhancement of Chest X-Rays

Task goal: Load a sample grayscale chest X-ray and apply the operations specified by the task: histogram equalization, `COLORMAP_JET`, color-balance adjustment, thresholding, logarithmic transformation, and fractional power-law/gamma transformation. The task asks for enhanced visual/structural visibility.

```
import cv2
import numpy as np

# =====
# 1. LOAD THE SAMPLE CHEST X-RAY
# =====

# Read the sample X-ray as grayscale because the task describes
# the input as a single-channel matrix.
xray = cv2.imread(
    "data/sample_xray.png",
    cv2.IMREAD_GRAYSCALE
)

# Stop if the image path is wrong or the file cannot be read.
if xray is None:
    print("Error: Could not load the X-ray image.")
    exit()

# Display the raw image for comparison.
cv2.imshow("Original X-Ray", xray)

# =====
# 2. HISTOGRAM EQUALIZATION
# =====

# Redistribute pixel intensities to improve contrast.
xray_equalized = cv2.equalizeHist(xray)

# Display the equalized image.
cv2.imshow("Equalized X-Ray", xray_equalized)

# =====
# 3. FALSE-COLOR / JET HEATMAP
# =====

# Convert the equalized grayscale image into a false-color heatmap.
```

```

heatmap = cv2.applyColorMap(
    xray_equalized,
    cv2.COLORMAP_JET
)

# Display the heatmap.
cv2.imshow("JET Heatmap", heatmap)

# =====
# 4. COLOR BALANCE
# =====

# Split the BGR heatmap into separate channels.
b, g, r = cv2.split(heatmap)

# Example channel scaling factors.
# CHANGE these values if a question gives specific values.
b_scale = 0.90
g_scale = 1.00
r_scale = 0.90

# Scale each channel and keep values inside 0-255.
b = np.clip(b * b_scale, 0, 255).astype(np.uint8)
g = np.clip(g * g_scale, 0, 255).astype(np.uint8)
r = np.clip(r * r_scale, 0, 255).astype(np.uint8)

# Recombine channels into a balanced BGR image.
balanced = cv2.merge([b, g, r])

# Display the balanced result.
cv2.imshow("Color Balanced", balanced)

# =====
# 5. THRESHOLDING
# =====

# Threshold value is a parameter; change it if the question specifies another value.
threshold_value = 180

# Keep only high-intensity/dense regions in a binary mask.
_, thresholded = cv2.threshold(
    xray_equalized,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

# Display the binary result.
cv2.imshow("Thresholded Dense Regions", thresholded)

```

```

# =====
# 6. LOGARITHMIC TRANSFORMATION
# =====

# Compute scaling constant so the log result fits into 0-255.
max_value = np.max(xray)
if max_value > 0:
    c = 255 / np.log(1 + max_value)
else:
    c = 1

# Apply log transformation to the raw underexposed X-ray.
# +1 prevents log(0).
log_xray = c * np.log(1 + xray)

# Convert back to 8-bit image format.
log_xray = np.uint8(log_xray)

# Display the log-transformed image.
cv2.imshow("Log Transformed X-Ray", log_xray)

# =====
# 7. POWER-LAW / GAMMA TRANSFORMATION
# =====

# Fractional gamma (< 1) brightens darker/mid-tone regions.
# The exact gamma should be changed if the exam gives a value.
gamma_value = 0.5

# Normalize intensities from 0-255 to 0-1.
normalized = xray / 255.0

# Apply the power-law transformation.
gamma_xray = np.power(normalized, gamma_value)

# Convert back to 0-255 8-bit image format.
gamma_xray = np.uint8(gamma_xray * 255)

# Display the gamma result.
cv2.imshow("Gamma Transformed X-Ray", gamma_xray)

# =====
# 8. SAVE OUTPUTS (OPTIONAL FOR EXAM, REQUIRED IN TASK SETUP)
# =====

# Save processed images using relative paths.
cv2.imwrite("output/xray_equalized.png", xray_equalized)

```

```

cv2.imwrite("output/xray_heatmap.png", heatmap)
cv2.imwrite("output/xray_threshold.png", thresholded)
cv2.imwrite("output/xray_log.png", log_xray)
cv2.imwrite("output/xray_gamma.png", gamma_xray)

```

```

# =====
# 9. CLOSE WINDOWS
# =====

# Wait for a key before closing.
cv2.waitKey(0)

# Close all OpenCV windows.
cv2.destroyAllWindows()

```

Task 1 function / argument dictionary

- `cv2.imread(filename, flags)` -> load the X-ray
- `cv2.equalizeHist(src)` -> histogram equalization
- `cv2.applyColorMap(src, colormap)` -> JET false-color mapping
- `cv2.split(image)` -> separate B/G/R channels
- `np.clip(array, 0, 255)` -> keep channel values valid
- `.astype(np.uint8)` -> 8-bit image type
- `cv2.merge([b,g,r])` -> combine B/G/R channels
- `cv2.threshold(src, thresh, maxval, type)` -> binary thresholding
- `np.max(xray)` -> maximum intensity
- `np.log(array)` -> log transformation
- `np.power(base, gamma)` -> gamma/power-law transformation
- `cv2.imwrite(filename, image)` -> save output image
- `cv2.imshow(window_name, image)` -> display
- `cv2.waitKey(delay)` -> keyboard wait
- `cv2.destroyAllWindows()` -> close windows

Task 1 exam modifications

- Different file: change the path in `cv2.imread()`.
 - Different threshold: change `threshold_value`.
 - Different gamma: change `gamma_value`.
 - Different color correction: change `b_scale`, `g_scale`, `r_scale`.
 - If only one operation is asked, you do not need the entire pipeline.
-

16. MASTER TASK 2 - Multi-Modal Cardiac Image Fusion

Task goal: Load one CT slice and its corresponding MRI slice, enhance each independently, convert them to colored heatmaps, combine them using weighted fusion with a heavier CT contribution, then apply logarithmic and power-law transformations and display CT, MRI, and fused results side-by-side.

```
import cv2
import numpy as np

# =====
# 1. LOAD THE MATCHED CT AND MRI SLICES
# =====

# Load CT slice as grayscale.
ct = cv2.imread(
    "data/ct.png",
    cv2.IMREAD_GRAYSCALE
)

# Load the corresponding MRI slice as grayscale.
mri = cv2.imread(
    "data/mri.png",
    cv2.IMREAD_GRAYSCALE
)

# Check that both files were loaded.
if ct is None or mri is None:
    print("Error: Could not load CT or MRI image.")
    exit()

# Check that the images have compatible dimensions.
if ct.shape != mri.shape:
    print("Error: CT and MRI dimensions do not match.")
    exit()

# =====
# 2. HISTOGRAM EQUALIZATION ON EACH MODALITY
# =====

# Equalize CT independently so its own intensity distribution is enhanced.
ct_equalized = cv2.equalizeHist(ct)

# Equalize MRI independently so its own intensity distribution is enhanced.
mri_equalized = cv2.equalizeHist(mri)
```

```

# =====
# 3. COLOR MAPPING
# =====

# Convert enhanced CT into a false-color heatmap.
ct_heatmap = cv2.applyColorMap(
    ct_equalized,
    cv2.COLORMAP_JET
)

# Convert enhanced MRI into a false-color heatmap.
# If the exam specifies a different colormap for MRI, replace JET here.
mri_heatmap = cv2.applyColorMap(
    mri_equalized,
    cv2.COLORMAP_JET
)

# =====
# 4. WEIGHTED FUSION
# =====

# CT receives the heavier contribution to preserve sharp anatomy.
alpha = 0.70

# MRI receives the lighter contribution to preserve its tissue information.
beta = 0.30

# Scalar offset; usually 0 when simply blending two images.
gamma_offset = 0

# Combine the two heatmaps using weighted addition.
fused = cv2.addWeighted(
    ct_heatmap,
    alpha,
    mri_heatmap,
    beta,
    gamma_offset
)

# =====
# 5. LOGARITHMIC TRANSFORMATION
# =====

# Calculate scaling constant for 0-255 output.
max_value = np.max(fused)
if max_value > 0:
    c = 255 / np.log(1 + max_value)

```

```

else:
    c = 1

# Apply log transformation to the fused image.
fused_log = c * np.log(1 + fused)

# Convert result to 8-bit format.
fused_log = np.uint8(fused_log)

# =====
# 6. POWER-LAW / GAMMA TRANSFORMATION
# =====

# Use a gamma value appropriate to the question.
# Example below is < 1; change it if the question gives a value.
gamma_value = 0.80

# Normalize to 0-1.
normalized = fused_log / 255.0

# Apply the power-law transformation.
fused_gamma = np.power(normalized, gamma_value)

# Convert back to 0-255 uint8.
fused_gamma = np.uint8(fused_gamma * 255)

# =====
# 7. COMPARATIVE ANALYSIS
# =====

# Put CT, MRI, and final fused result next to each other.
comparison = np.hstack((
    ct_heatmap,
    mri_heatmap,
    fused_gamma
))

# Display the three views side-by-side.
cv2.imshow("CT | MRI | Fused", comparison)

# =====
# 8. SAVE OUTPUTS (OPTIONAL FOR EXAM)
# =====

cv2.imwrite("output/ct_equalized.png", ct_equalized)
cv2.imwrite("output/mri_equalized.png", mri_equalized)
cv2.imwrite("output/ct_heatmap.png", ct_heatmap)
cv2.imwrite("output/mri_heatmap.png", mri_heatmap)

```

```
cv2.imwrite("output/fused.png", fused)
cv2.imwrite("output/fused_final.png", fused_gamma)
cv2.imwrite("output/comparison.png", comparison)
```

```
# =====
# 9. CLOSE WINDOWS
# =====
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Task 2 function / argument dictionary

- `cv2.imread(filename, flags)` -> load CT/MRI grayscale slices
- `image.shape` -> image dimensions; useful for compatibility checking
- `cv2.equalizeHist(src)` -> independent histogram equalization
- `cv2.applyColorMap(src, colormap)` -> false-color heatmap
- `cv2.addWeighted(src1, alpha, src2, beta, gamma)` -> weighted fusion
- `np.max(fused)` -> maximum intensity used for log scaling
- `np.log(array)` -> logarithmic transform
- `np.power(base, gamma)` -> power-law transform
- `np.hstack(...)` -> horizontal side-by-side comparison
- `cv2.imwrite(filename, image)` -> save result

Task 2 exam modifications

- If CT should dominate: make `alpha` larger than `beta`.
- If MRI should dominate: make `beta` larger (only when instructed).
- Change `gamma` if the question specifies it.
- Change `colormap` if the question names one.
- Ensure the images are corresponding/aligned and compatible for fusion.

17. MASTER TASK 3 - Real-Time Echocardiogram Video Analysis

Task goal: Open the provided ultrasound `.mp4`, process every frame in a `while` loop, apply histogram equalization, JET heatmap, color balance, logarithmic transformation, and power-law transformation, then display raw and enhanced frames side-by-side.

```

import cv2
import numpy as np

# =====
# 1. OPEN THE ULTRASOUND VIDEO FILE
# =====

# Open the provided .mp4 file rather than a webcam.
cap = cv2.VideoCapture("data/echo.mp4")

# Check that the video file was opened successfully.
if not cap.isOpened():
    print("Error: Could not open ultrasound video.")
    exit()

# =====
# 2. PROCESS THE VIDEO FRAME BY FRAME
# =====

while True:

    # Read the next frame.
    # ret = True/False success flag
    # frame = current image frame
    ret, frame = cap.read()

    # Stop when no frame is available, usually at end of video.
    if not ret:
        break

    # =====
    # 3. GRAYSCALE CONVERSION
    # =====

    # Convert BGR frame to single-channel grayscale.
    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # =====
    # 4. HISTOGRAM EQUALIZATION
    # =====

    # Improve ultrasound contrast by redistributing intensities.
    equalized = cv2.equalizeHist(gray)

```

```

# =====
# 5. JET FALSE-COLOR MAPPING
# =====

# Convert equalized grayscale frame into a JET heatmap.
heatmap = cv2.applyColorMap(
    equalized,
    cv2.COLORMAP_JET
)

# =====
# 6. COLOR BALANCE
# =====

# Split the heatmap into OpenCV's BGR channels.
b, g, r = cv2.split(heatmap)

# Example balancing factors.
# Change these if the question specifies exact values.
b_scale = 0.90
g_scale = 1.00
r_scale = 0.90

# Scale and clip each channel to valid 8-bit range.
b = np.clip(b * b_scale, 0, 255).astype(np.uint8)
g = np.clip(g * g_scale, 0, 255).astype(np.uint8)
r = np.clip(r * r_scale, 0, 255).astype(np.uint8)

# Recombine channels.
balanced = cv2.merge([b, g, r])

# =====
# 7. LOGARITHMIC TRANSFORMATION
# =====

# Calculate scaling constant to map log output to 0-255.
max_value = np.max(balanced)
if max_value > 0:
    c = 255 / np.log(1 + max_value)
else:
    c = 1

# Apply log transformation to reveal darker regions.
log_image = c * np.log(1 + balanced)

# Convert to uint8 for display.
log_image = np.uint8(log_image)

```

```

# =====
# 8. POWER-LAW / GAMMA TRANSFORMATION
# =====

# Gamma > 1 suppresses brighter values.
# This matches the task's goal of suppressing bright backscatter.
gamma_value = 2.0

# Normalize image to 0-1.
normalized = log_image / 255.0

# Apply gamma transformation.
gamma_image = np.power(
    normalized,
    gamma_value
)

# Convert back to 0-255 uint8.
gamma_image = np.uint8(gamma_image * 255)

# =====
# 9. RAW VS ENHANCED SIDE-BY-SIDE DISPLAY
# =====

# Raw frame and enhanced frame have compatible dimensions.
comparison = np.hstack((
    frame,
    gamma_image
))

# Show raw and enhanced video at the same time.
cv2.imshow(
    "Raw Ultrasound | Enhanced Ultrasound",
    comparison
)

# =====
# 10. EXIT CONTROL
# =====

# Wait briefly so the video continues playing.
# Press q to exit manually.
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# =====
# 11. RELEASE RESOURCES

```

```
# =====  
  
# Release the video file/device.  
cap.release()  
  
# Close all windows.  
cv2.destroyAllWindows()
```

Task 3 function / argument dictionary

- `cv2.VideoCapture(source)` -> open video
- `cap.isOpened()` -> check source
- `cap.read()` -> get (ret, frame)
- `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)` -> grayscale conversion
- `cv2.equalizeHist(gray)` -> histogram equalization
- `cv2.applyColorMap(equalized, cv2.COLORMAP_JET)` -> false-color map
- `cv2.split()` / `cv2.merge()` -> color balance
- `np.clip()` / `.astype()` -> safe 8-bit channel values
- `np.log()` -> log transformation
- `np.power()` -> gamma transformation
- `np.hstack()` -> side-by-side display
- `cv2.imshow()` -> display video output
- `cv2.waitKey(1)` -> short delay + keyboard detection
- `cap.release()` -> release video
- `cv2.destroyAllWindows()` -> close windows

Task 3 exam modifications

- Video file path changes: update `cv2.VideoCapture("...").`
- Webcam version, only if specifically asked: `cv2.VideoCapture(0).`
- Change gamma according to requirement.
- Change color-balance coefficients if provided.
- Keep the `while True -> read -> if not ret -> process -> imshow -> waitKey` pattern.

18. MASTER FUNCTION QUICK REFERENCE

Image input

```
cv2.imread(filename, flags)
```

- `filename` = path
- `flags` = image-read mode
- common grayscale flag: `cv2.IMREAD_GRAYSCALE`

Histogram

```
cv2.calcHist(images, channels, mask, histSize, ranges)
```

- `images` = list of images
- `channels` = list of channel indices
- `mask` = mask or `None`
- `histSize` = number of bins
- `ranges` = value range

Histogram equalization

```
cv2.equalizeHist(src)
```

- `src` = single-channel 8-bit grayscale image

Color conversion

```
cv2.cvtColor(src, code)
```

- `src` = input image
- `code` = conversion code
- grayscale conversion: `cv2.COLOR_BGR2GRAY`

Color map

```
cv2.applyColorMap(src, colormap)
```

- `src` = input grayscale image in this workflow
- `colormap` = e.g. `cv2.COLORMAP_JET`

Threshold

```
cv2.threshold(src, thresh, maxval, type)
```

- `src` = input image
- `thresh` = cutoff
- `maxval` = output value for pixels passing rule
- `type` = thresholding mode

Weighted fusion

```
cv2.addWeighted(src1, alpha, src2, beta, gamma)
```

- `src1` = first image
- `alpha` = first-image weight
- `src2` = second image
- `beta` = second-image weight
- `gamma` = scalar offset

Color channels

```
cv2.split(image)  
cv2.merge([b, g, r])
```

- OpenCV color order = BGR

Mathematical transforms

```
np.log(array)  
np.power(base, exponent)  
np.max(array)  
np.clip(array, min_value, max_value)
```

Array concatenation

```
np.hstack((array1, array2, ...))
```

- horizontal concatenation

Video

```
cv2.VideoCapture(source)  
cap.isOpened()
```

```
cap.read()
cap.release()
```

Display

```
cv2.imshow(window_name, image)
cv2.waitKey(delay)
cv2.destroyAllWindows()
```

Save

```
cv2.imwrite(filename, image)
```

19. LAB 2 "IF SIR CHANGES THE QUESTION" CHEAT TABLE

Question asks	Use / change
Read grayscale image	<code>cv2.imread(..., cv2.IMREAD_GRAYSCALE)</code>
Read color image	<code>cv2.imread(..., cv2.IMREAD_COLOR)</code>
Display image	<code>cv2.imshow('name', image)</code>
Histogram	<code>cv2.calcHist([gray], [0], None, [256], [0,256])</code>
Histogram Equalization	<code>cv2.equalizeHist(gray)</code>
Log transform	<code>c * np.log(1 + image)</code> then <code>np.uint8(...)</code>
Gamma / power-law	<code>np.power(image/255.0, gamma) * 255</code>
Gamma < 1	brighten
Gamma > 1	darken / suppress brighter values
Piecewise transform	separate intensity ranges with different rules

Question asks	Use / change
Binary threshold	<code>cv2.threshold(image, T, 255, cv2.THRESH_BINARY)</code>
False-color heatmap	<code>cv2.applyColorMap(gray, cv2.COLORMAP_JET)</code>
Adjust BGR channels	<code>cv2.split</code> -> <code>scale/clip</code> -> <code>cv2.merge</code>
Fuse CT + MRI	<code>cv2.addWeighted(ct, alpha, mri, beta, gamma)</code>
CT heavier weight	choose <code>alpha > beta</code> if requested
Side-by-side images	<code>np.hstack((img1, img2, ...))</code>
Open video file	<code>cv2.VideoCapture('data/video.mp4')</code>
Read next video frame	<code>ret, frame = cap.read()</code>
End-of-video check	<code>if not ret: break</code>
Raw + enhanced stream	<code>np.hstack((frame, enhanced))</code>
Quit video	<code>if cv2.waitKey(1) & 0xFF == ord('q'): break</code>
Release video	<code>cap.release()</code>

20. THE 10 CODE PATTERNS TO MEMORIZE FIRST

Pattern 1 - Read grayscale image

```
image = cv2.imread("path", cv2.IMREAD_GRAYSCALE)
```

Pattern 2 - Histogram equalization

```
equalized = cv2.equalizeHist(image)
```

Pattern 3 - Histogram

```
hist = cv2.calcHist([image], [0], None, [256], [0, 256])
```

Pattern 4 - JET color map

```
heatmap = cv2.applyColorMap(image, cv2.COLORMAP_JET)
```

Pattern 5 - Threshold

```
_, binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
```

Pattern 6 - Log

```
c = 255 / np.log(1 + np.max(image))  
log_img = np.uint8(c * np.log(1 + image))
```

Pattern 7 - Gamma

```
gamma_img = np.uint8(np.power(image / 255.0, gamma) * 255)
```

Pattern 8 - Fusion

```
fused = cv2.addWeighted(img1, alpha, img2, beta, gamma_offset)
```

Pattern 9 - Video loop

```
cap = cv2.VideoCapture("video.mp4")  
while True:  
    ret, frame = cap.read()  
    if not ret:  
        break  
    # process frame
```

Pattern 10 - Side-by-side

```
comparison = np.hstack((raw, enhanced))
```

21. FAST MCQ REVISION

- Point transformation: output intensity depends on the input intensity at that pixel.
- Log transformation: expands dark intensities, compresses bright intensities.
- Gamma < 1: brightens.
- Gamma > 1: darkens / suppresses brighter values.
- Gamma = 1: approximately unchanged.
- Piecewise transformation: different rules for different intensity intervals.
- Histogram: distribution/count of pixels over intensity levels.
- Histogram does not preserve spatial location information.
- Histogram equalization: redistributes intensity values to improve contrast.
- `cv2.equalizeHist()` expects a single-channel 8-bit grayscale image in this workflow.
- `cv2.applyColorMap()` creates a false-color representation.
- Lab 2 uses `COLORMAP_JET` for the explicitly stated heatmap operations in Tasks 1 and 3.
- OpenCV color channel order: BGR.
- `cv2.threshold()` returns `(threshold_used, output_image)`.
- `cv2.addWeighted()` performs weighted addition/fusion.
- `alpha` = first-image weight.
- `beta` = second-image weight.
- `gamma` in `addWeighted` = scalar offset; it is NOT the gamma exponent used in power-law transformation.
- `cv2.VideoCapture("file.mp4")` opens a video file.
- `cap.read()` returns `(ret, frame)`.
- `ret == False` normally means no frame was obtained/end of video.
- `cv2.waitKey(0)` = indefinite wait; `cv2.waitKey(1)` = short delay suitable for video playback.

22. LAST-MINUTE EXAM MENTAL MAP

TASK 1 - X-RAY

```
read grayscale
-> equalizeHist
-> applyColorMap(JET)
-> color balance
-> threshold
-> log
-> gamma (< 1)
-> display/save
```

TASK 2 - CT + MRI

```
read CT + MRI
-> equalize each independently
-> color maps
-> addWeighted (CT heavier, MRI lighter)
```

```
-> log
-> gamma
-> hstack / display CT | MRI | fused
```

TASK 3 - ULTRASOUND VIDEO

VideoCapture

```
-> while True
-> read frame
-> BGR -> grayscale
-> equalizeHist
-> JET
-> color balance
-> log
-> gamma (> 1 for suppression example)
-> hstack(raw, enhanced)
-> imshow
-> waitKey(1)
-> repeat
-> release
```

23. EXAM-SAFE REMINDERS

LOG:

```
np.log(1 + image)
```

GAMMA:

```
np.power(image / 255.0, gamma)
```

THRESHOLD:

```
cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
```

JET:

```
cv2.applyColorMap(image, cv2.COLORMAP_JET)
```

FUSION:

```
cv2.addWeighted(img1, alpha, img2, beta, gamma_offset)
```

VIDEO:

```
ret, frame = cap.read()
if not ret:
    break
```

BGR:

```
b, g, r = cv2.split(image)
```

Source alignment

This sheet follows the structure of the provided Lab 4 cheat sheet: topic/purpose, commented code, function/argument dictionary, and exam modifications. The Lab 2 task-specific sections are based on the operations explicitly listed in the supplied Lab 2 Task PDF.